BÖLÜM 5: ÖLÇEKLENEBILIRLIK VE ESNEKLIK — İZDIHAMI YÖNETMEK 💣 9 dk • DERS 20

Parça 20: Hız Sınırlama (Rate Limiting) ve Trafik Şekillendirme: Kalenizi Korumak

Veri kazıyıcılardan (scrapers), kontrolden çıkmış döngülerden ve hacimsel saldırılardan sağ çıkma mimarisi.

> 🔥 **SAHADAN YAŞANMIŞ KRİZ HİKAYESİ**
>
> Bir hava durumu API'sinin cömert bir ücretsiz planı vardı. Güneşli bir Salı sabahı, akıllı bahçe sulama cihazları üreten bir firmanın junior geliştiricisi toprağın nem durumuna göre hava tahminini çeken bir Python betiği yazdı. Ancak API çağrısını içine hiçbir `sleep` beklemesi koymadan sonsuz bir `while True:` döngüsüne yerleştirdi! Bu güncelleme 500 akıllı sulama cihazına yüklendi ve hava durumu şirketine saniyede 45.000 istek yağmaya başladı. Hava durumu API'sinde hız sınırlama yoktu. 60 saniye içinde veritabanı çöktü, gerçek mobil kullanıcılar 504 Gateway Timeout almaya başladı ve şirketin iki saatlik AWS internet çıkış (egress) faturası 4.500 doları aştı. Tüm şirket tek bir junior yazılımcının hatası yüzünden rehin alınmıştı.

Zihniyet Dönüşümü: Kod Yazarlığından Mimarlığa

Dağıtık dünyada sınırsız erişim sunmak varoluşsal bir intihardır. Dost müşteriler bile hatalı yeniden deneme döngüleriyle veya agresif botlarla sizi istemeden çökertebilir. Hız sınırlama (Rate Limiting) sadece parayla API paketi satanların kullandığı ticari bir sayaç değildir; mimarinin temel bağışıklık sistemidir. Usta bir mimar trafiği dış sınırda şekillendirir ve iç kaynakları ne pahasına olursa olsun korur.

Temel Mimari İlkeler ve Zihinsel Modeller

Klasik Hız Sınırlama Algoritmaları

- **Token Bucket (Jeton Kovası):** Kovaya sabit hızla jeton damlar. Her istek bir jeton harcar. Ani trafik patlamalarına (burst) kova kapasitesi kadar izin verirken ortalama hızı korur. (API standartı).
- **Leaky Bucket (Sızıntılı Kova):** İstekler kovaya dolar ve tabandaki delikten sabit hızla akar. Düzensiz trafiği sabit hızda pürüzsüz bir akışa dönüştürür.
- **Kayan Pencere Sayacı (Sliding Window Counter):** İstek sayısını geçmiş ve şimdiki zaman diliminin ağırlıklı toplamıyla hesaplar. Düşük bellek tüketimi ve %99.9 doğruluk sunar.

Ölçeklenebilir Hız Sınırlama Mimarisi

Dağıtık hız sınırlamayı ASLA ilişkisel veritabanına sorgu atarak yapmayın! Atomik Lua betikleriyle çalışan bellek içi bir Redis kümesi veya NGINX/Cloudflare/Kong gibi dış ağ geçitleri kullanın.

Kaçınılması Gereken Anti-Pattern'ler ve Tuzaklar

- **Yalnızca IP'ye Göre Kısıtlama:** Tek bir NAT ağ geçidinin arkasındaki 5.000 kişilik bir üniversiteyi veya şirketi tek bir kullanıcının hatası yüzünden toptan bloklamak.
- **Retry-After Başlığını Unutmak:** `HTTP 429 Too Many Requests` dönerken istemciye ne kadar beklemesi gerektiğini söyleyen `Retry-After: 30` başlığını vermemek; çaresiz istemcilerin daha da hızlı istek atmasına yol açmak.

- **Veritabanı Tabanlı Sayaç:** Her istekte SQL'e `INSERT INTO istek_loglari` yazmak; yani gelen bir DDoS atağında hız sınırlayıcının çalışabilmek için veritabanını bizzat kendisinin çökertmesi.

> 🎯 **MİMARIN EYLEM PLANI: BU HAFTAKİ SOMUT ADIMLAR**
>
> - **1. Adım:** Katmanlı hız sınırlama kur: Anonim kullanıcılara IP bazlı, giriş yapmış kullanıcılara API anahtarı veya Kullanıcı ID'si bazlı limit koy.
> - **2. Adım:** Standart RFC 6585 başlıklarını uygula: `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset`.
> - **3. Adım:** Hız sınırlamayı en dış sınırda (CDN / Tersine Vekil) konumlandır ki engellenen istekler arka uç sunucularının CPU'suna asla dokunmasın.

Mimari Çıkarımlar ve Hile Tablosu

- API'nizde hız sınırlama yoksa, sisteminizin ayakta kalması en dikkatsiz istemcinizin insafına kalmıştır.
- Token Bucket, esneklik ve patlama toleransı için endüstri standardı algoritmadır.
- Zararlı trafiği kalenin en dış kapısında durdurun; iç odalara girmesine izin vermeyin.

> 🎙️ **UDEMY & NOTEBOOKLM TARTIŞMA VE ÖDEV SORUSU**
>
> **Ders Değerlendirmesi ve Ödev:** Birisi şu an API'nize 10 saniyede 10.000 istek gönderse ne olur? Sisteminiz onu engeller mi, yoksa 10.000 sorgunun hepsini veritabanına çalıştırmayı dener mi?